AI 388 / CS 388 Natural Language Processing

Module 3: Word Embeddings

Topics 10, 11

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Full derivations** (added Oct 20): every formula derived step by step with nothing assumed, plus low-dimensional by-hand examples of the algorithms worked in lecture and "conceptual checks" for Viterbi/CKY.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide **1**

10 Word Embeddings: Skip-gram, Properties, Bias **3**

10.1 Skip-gram model 3
10.2 Full derivations 4
10.3 Other embedding methods (3.1.1) 6
10.4 Properties of embeddings 6
10.5 Bias in word embeddings (3.1.2) 7
10.6 Long skip-gram problems 8
10.7 Practice 9

11 Deep Averaging Networks (Assignment 2) **10**

11.1 Model 10
11.2 Assignment 2 implementation details 10
11.3 Derivations 11
11.4 Limitations (exam favorite) 11
11.5 Past exam problems 12
11.6 Practice 13

TOPIC 10

Word Embeddings: Skip-gram, Properties, Bias

Sources: Lectures 3.0.0 Word Embeddings, 3.1.0 Skip-gram, 3.1.1 Other Word Embedding Methods, 3.1.2 Bias in Word Embeddings; slides seg-17–seg-20; Exercises 3.0, 3.1; Fall 2022 Q1(1), Q1(5), Q1(11–13), Q2(a), Q5; Fall 2023 Q1(4), Q3.

In plain words

One-hot/BoW vectors make every word equally far from every other: *movie* and *film* are orthogonal, so "movie was good" and "film is great" look unrelated. **Word embeddings** are short dense vectors (50–300 dims) where words used in similar contexts get similar vectors. **Distributional hypothesis** (Firth 1957): "You shall know a word by the company it keeps." Skip-gram learns these vectors from raw text, with no labels, by making each word's vector good at predicting the words around it.

10.1 Skip-gram model

Skip-gram (word2vec, Mikolov et al. 2013)

Input: a large raw corpus. **Output:** for each word *type* w, a word vector $\mathbf{v}_w$ and a context vector $\mathbf{c}_w$, both in $\mathbb{R}^d$. **Hyperparameters:** dimension d (50–300) and window size k.
Training pairs: for every token, pair it (as the *word*) with every token up to k positions away (as the *context*).

$$P(\text{context} = y \mid \text{word} = x) = \frac{\exp(\mathbf{v}_x \cdot \mathbf{c}_y)}{\sum_{y' \in V} \exp(\mathbf{v}_x \cdot \mathbf{c}_{y'})}$$

The denominator sums over the **whole vocabulary**. This is multiclass LR where the "features" are $\mathbf{v}_x$ and the "class weights" are the context vectors.
Parameters: $V \in \mathbb{R}^{|V| \times d}$ (word vectors) and $C \in \mathbb{R}^{|V| \times d}$ (context vectors), for $2|V|d$ in total.
Objective: maximize $\sum_{(x,y) \in \text{pairs}} \log P(y|x)$ by SGD from **random** initialization. This is an "impossible" problem: a word has many different contexts, so you can never get every pair to probability 1 (unlike classification).

Matrix view (3.1.1): $P(\text{context} \mid x) = \text{softmax}(W\mathbf{v}_x)$, where row y of W is $\mathbf{c}_y$. The name "skip-gram" comes from predicting words that skip over intermediate words (window $k > 1$).

Exam procedure: skip-gram probabilities

1. Compute $\mathbf{v}_x \cdot \mathbf{c}_y$ for **every** y in the vocabulary, including $y = x$ and words never seen next to x.
2. Exponentiate (use the question's value for e if given, e.g. $e = 3$; $e^0 = 1$).
3. Divide each by the sum.

To find *optimal* vectors: make the dot products with observed contexts large and the rest very negative ($\pm\infty$ in idealized questions). If a word has two equally frequent contexts, the optimum is probability $\frac{1}{2}$ each.

Worked example: lecture / Exercise 3.1 Q1 ("I saw", $e = 3$)

$\mathbf{v}_I = [1,0]$, $\mathbf{v}_{saw} = [0,1]$, $\mathbf{c}_{saw} = [1,0]$, $\mathbf{c}_I = [0,1]$. Pairs (k=1): (I, saw), (saw, I). $P(\cdot \mid saw)$: $\mathbf{v}_{saw} \cdot \mathbf{c}_I = 1 \to e^1 = 3$; $\mathbf{v}_{saw} \cdot \mathbf{c}_{saw} = 0 \to 1$. So $P(I|saw) = 3/4$, $P(saw|saw) = 1/4$. Note $\mathbf{c}_I$ sits on top of $\mathbf{v}_{saw}$: word space and context space are different (roughly rotations of each other). A word's vector is close to its contexts' *context* vectors, not to their word vectors.

Canvas Exercise 3.1 Q2–Q4

Q2: with $\mathbf{v}_I = [1,0]$, $\mathbf{v}_{saw} = [0,1]$, the optimal context vectors are

Answer: $\mathbf{c}_{saw} = [1,-\infty]$, $\mathbf{c}_I = [-\infty,1]$. This gives $P(saw|I) \to 1$ and $P(I|saw) \to 1$.

Q3: corpus "the dog", "the cat", "a dog" with $k = 1$.

Answer: 6 pairs (two per sentence).

Q4: contexts fixed at $\mathbf{c}_a = \mathbf{c}_{the} = (1,0)$, $\mathbf{c}_{cat} = \mathbf{c}_{dog} = (0,1)$. "the" occurs with cat and dog once each, so the best is $P = \frac{1}{2}$ each:

Answer: $\mathbf{v}_{the} = (-\infty, 0)$ or $(0, \infty)$.

Counting skip-gram pairs

A sentence of L tokens with window k (and $L > k$) gives $2\left(kL - \frac{k(k+1)}{2}\right)$ ordered pairs. Each unordered neighbor pair appears in both directions.

Midterm Fall 2022 Q1(1)

All skip-gram (word, context) pairs for *the cat ran away*, window ± 2.

Official answer: C: [the, cat], [the, ran], [cat, the], [cat, ran], [cat, away], [ran, the], [ran, cat], [ran, away], [away, cat], [away, ran]. That's $2(2 \cdot 4 - 3) = 10$ pairs. Pairs are single words, both directions, and nothing farther than 2 away (the/away is excluded).

10.2 Full derivations

A. Counting pairs: $2\left(kL - \frac{k(k+1)}{2}\right)$

1. Count unordered neighbor pairs by their distance $\delta \in \{1, \ldots, k\}$. Pairs at distance δ start at positions $1..L-\delta$: there are $L - \delta$ of them.
2. Total unordered: $\sum_{\delta=1}^{k}(L-\delta) = kL - \sum_{\delta=1}^{k}\delta = kL - \frac{k(k+1)}{2}$ (needs $L > k$; otherwise use $\binom{L}{2}$-style counting).
3. Each unordered pair gives two (word, context) pairs, one per direction. Check: "the cat ran away", $k = 2$: $2(8-3) = 10$. ✓

B. Skip-gram gradients (same algebra as multiclass LR)

One training pair (word x, observed context y). Scores $s_{y'} = \mathbf{v}_x \cdot \mathbf{c}_{y'}$ for every vocabulary word y'.

1. $\mathcal{L} = -\log P(y|x) = -\mathbf{v}_x \cdot \mathbf{c}_y + \log \sum_{y'} \exp(\mathbf{v}_x \cdot \mathbf{c}_{y'})$.
2. With respect to the word vector: $\dfrac{\partial \mathcal{L}}{\partial \mathbf{v}_x} = -\mathbf{c}_y + \sum_{y'} P(y'|x)\,\mathbf{c}_{y'}$ (derivative of $\log \sum \exp$ gives probability-weighted inputs, as in Topic 6). SGD moves $\mathbf{v}_x$ *toward* the observed context vector

and *away from* the model's average context vector.

3. With respect to each context vector: $\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{y'}} = \left(P(y'|x) - \mathbb{1}[y' = y]\right) \mathbf{v}_x$. The observed context is pulled toward $\mathbf{v}_x$; every other context is pushed away in proportion to its probability.
4. **Why similar words end up close (the distributional hypothesis, mechanically).** If *movie* and *film* both occur with *watched*, both $\mathbf{v}_{\text{movie}}$ and $\mathbf{v}_{\text{film}}$ are pulled toward $\mathbf{c}_{\text{watched}}$, so they drift toward each other.
5. Cost: the sum in steps 2–3 runs over the whole vocabulary, so one update is $O(|V|d)$. That's the motivation for hierarchical softmax and negative sampling.

C. Why the objective can't reach probability 1 ("impossible problem"), and what the optimum is

1. Group the pairs by word x. Let n_{xy} be how often context y appears with x, and $n_x = \sum_y n_{xy}$.
2. The terms for x are $\sum_y n_{xy} \log P(y|x)$, with $\sum_y P(y|x) = 1$.
3. If the vectors are expressive enough to give any distribution, the maximizer is $P(y|x) = n_{xy}/n_x$ (the relative frequencies; the same count-and-normalize result as n-gram MLE, Topic 12).
4. So a word seen with two contexts equally often gets $\frac{1}{2}$ each, never 1. Exercise 3.1 Q4 is exactly this case.

D. Negative sampling objective, piece by piece

1. Label a pair (word w, context c) as real ($+1$) or fake (-1). Model: $P(\text{real} \mid w, c) = \sigma(\mathbf{v}_w \cdot \mathbf{c}_c)$, binary LR with score $\mathbf{v}_w \cdot \mathbf{c}_c$.
2. $P(\text{fake} \mid w, c) = 1 - \sigma(s) = \sigma(-s)$ (shown in Topic 3).
3. For one real pair plus k sampled fake contexts $c'_1..c'_k$, the log-likelihood is $\log \sigma(\mathbf{v}_w \cdot \mathbf{c}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_w \cdot \mathbf{c}_{c'_i})$.
4. Cost per pair: $k + 1$ dot products instead of $|V|$.

E. Smaller facts, derived

- **Hierarchical softmax.** A balanced binary tree with $|V|$ leaves has depth $\lceil \log_2 |V| \rceil$, so one prediction is $\approx \log_2 |V|$ binary decisions. A binary tree with $|V|$ leaves has exactly $|V| - 1$ internal nodes (each merge of two subtrees adds one node and reduces the count of subtrees by one: $|V| \to 1$ takes $|V| - 1$ merges). One vector per internal node, so the parameter count stays about the same.
- **PMI.** $\text{PMI}(w, c) = \log \frac{P(w,c)}{P(w)P(c)}$ with $P(w,c) = \frac{\#(w,c)}{|D|}$, $P(w) = \frac{\#w}{|D|}$, $P(c) = \frac{\#c}{|D|}$. It is 0 when w and c co-occur exactly as often as chance predicts, positive when more often.
- **Debiasing projection.** With a unit gender direction $\mathbf{g}$ ($\|\mathbf{g}\| = 1$): $\text{proj}_{\mathbf{g}}(\mathbf{w}) = (\mathbf{w} \cdot \mathbf{g})\,\mathbf{g}$ and $\mathbf{w}' = \mathbf{w} - (\mathbf{w} \cdot \mathbf{g})\mathbf{g}$. Check: $\mathbf{w}' \cdot \mathbf{g} = \mathbf{w} \cdot \mathbf{g} - (\mathbf{w} \cdot \mathbf{g})(\mathbf{g} \cdot \mathbf{g}) = 0$. Example: $\mathbf{g} = (1, 0)$, homemaker $= (0.8, 0.3) \Rightarrow (0, 0.3)$. Its other coordinates (and so its neighbors) are untouched, which is why bias survives.
- **Cosine = dot after normalizing.** $\frac{\mathbf{a}}{\|\mathbf{a}\|} \cdot \frac{\mathbf{b}}{\|\mathbf{b}\|} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$.

10.3 Other embedding methods (3.1.1)

method	key idea
CBOW	Flip skip-gram: **sum the context words' embeddings** and predict the center word: $P(w\|\text{ctx}) = \text{softmax}(W(\mathbf{c}_{w_{-1}} + \mathbf{c}_{w_{+1}}))$. Same parameter count; performs similarly.
Hierarchical softmax	Replace the $\|V\|$-way softmax with a binary tree (e.g. Huffman) of binary decisions: $O(\log \|V\|)$ dot products per prediction instead of $O(\|V\|)$. Same number of parameters ($\|V\| - 1$ internal nodes).
Skip-gram w/ negative sampling (SGNS)	Binary classification of (word, context) pairs as real or fake: score $\mathbf{v}_w \cdot \mathbf{c}_c$ through a sigmoid. Real pairs come from data; k negatives by sampling random contexts. Most common in practice (efficient). max $\log \sigma(\mathbf{v}_w \cdot \mathbf{c}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_w \cdot \mathbf{c}_{c_i'})$.
Matrix factorization	Levy & Goldberg (2014): SGNS implicitly factorizes the $\|V\| \times \|V\|$ matrix with entries $\text{PMI}(w, c) - \log k$, where $\text{PMI} = \log \frac{P(w,c)}{P(w)P(c)}$ (negatives from the unigram distribution; weighted factorization).
GloVe	Regression on the **log co-occurrence counts** matrix using word/context vectors plus bias constants. Cost is quadratic in $\|V\|$ but constant in corpus size. Most widely used static embeddings (A2 uses GloVe 50d/300d).
fastText	SGNS where a word = sum of its **character n-gram** vectors (3–6-grams) $\Rightarrow$ can embed unseen words.
Contextual / subword	Pre-trained models (ELMo, BERT) split rare words into subwords and compute *context-dependent* embeddings with RNNs/Transformers (Topic 16, Topic 17).

Midterm Fall 2022 Q2(a)

Difference between skip-gram and CBOW?

Official answer: CBOW uses all surrounding context words at once to predict the current word; skip-gram uses the input word to predict each context word one at a time.

10.4 Properties of embeddings

- Similar words cluster: movie $\approx$ film. This helps a downstream model generalize to words not in its training set ("film is great" after training on "movie was good").
- **Antonyms can be close:** *good*/*bad* share contexts ("good food", "bad food"). Nothing forces opposites to have opposite vectors. This is a known problem for sentiment.
- Words that co-occur but behave differently (*I*/*saw*, *baseball*/*bat*) are *not* necessarily similar. Similarity comes from *substitutability* (sharing contexts), not co-occurrence.
- Words never sharing contexts (*tuck*/*ship*) have mostly unrelated embeddings.
- **Polysemy:** one vector per word type for all senses (*bark* of a tree vs. a dog; *film* reel vs. movie).
- Evaluation: *intrinsic* (word similarity) vs. *extrinsic* (downstream task performance, which is what we care about).

Canvas Exercise 3.0

Q1 true statements:

Answer: similar meanings $\Rightarrow$ similar embeddings; opposite meanings *might* have similar embeddings; words never in the same context have mostly unrelated embeddings. False: "opposites have opposite embeddings"; "dissimilar words that co-occur (baseball/bat) have similar embeddings".

Q2 shortcomings:

Answer: polysemous words get one embedding; embeddings can encode biases. (Tokenization cost is not a real issue.)

10.4.1 Similarity measures

Dot product vs. cosine

$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ vs. $\cos(\mathbf{a}, \mathbf{b}) = \dfrac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$. The dot product grows with vector length, so long (often frequent) vectors look similar to everything. Cosine ignores length. After normalizing all vectors to length 1, dot product = cosine.

Midterm Fall 2022 Q1(11)–(13)

good $= [0, 0, 3]$, bad $= [0, 1, 3]$, the $= [10, 10, 10]$. Dot products: good·bad $= 9$, good·the $= 30$, bad·the $= 40$, so most similar by dot product:

Official answer: C (bad, the). Cosine: $9/(3\sqrt{10}) = 0.949$; $30/(3 \cdot 17.32) = 0.577$; $40/(3.162 \cdot 17.32) = 0.730$, so

Official answer: A (good, bad). Normalized dot product = cosine:

Official answer: A.

10.5 Bias in word embeddings (3.1.2)

- Web text encodes societal biases, and embeddings absorb them at the very first stage of the pipeline.
- Bolukbasi et al. (2016), "Man is to Computer Programmer as Woman is to Homemaker?": take the *she*−*he* direction (a gender axis). Occupations most aligned with each end reflect stereotypes (*homemaker* at the extreme *she* end). Similar effects show up for race and religion.
- **Debiasing:** identify a gender subspace from gendered pairs (she/he, woman/man, ...). For words that should be neutral (*homemaker*), project onto the subspace and **subtract that projection**: $\mathbf{w}' = \mathbf{w} - \text{proj}_{\text{gender}}(\mathbf{w})$. This "neutralizes" the word along the gender axis.
- **Limitation:** bias is pervasive in the geometry. *homemaker* stays near other words (*child*, *caring*) that still carry gender associations, so neutralizing one direction doesn't remove it. The method also assumes a gender binary. This is an open problem.

Midterm Fall 2022 Q1(5)

What bias do Bolukbasi et al. remove?

Official answer: A: correlation with a he–she axis in embedding space.

Midterm Fall 2023 Q1(4)

True of Bolukbasi debiasing?

Official answer: C: it "neutralizes" words with respect to a gender subspace. Not A (no vocabulary removal), not B (dimensionality isn't reduced), not D ("homemaker" can still be similar to historically gendered concepts).

10.6 Long skip-gram problems

Midterm Fall 2023 Q3 ($e = 3$; $e^{10} \approx \infty$)

$\mathbf{v}_1 = (0,0)$, $\mathbf{v}_2 = (1,1)$, $\mathbf{v}_3 = (-1,1)$; $\mathbf{c}_1 = (0,0)$, $\mathbf{c}_2 = (0,1)$, $\mathbf{c}_3 = (1,0)$.
(a) Distribution for $\mathbf{v}_1$: all dot products 0, so

Official answer: $(1/3, 1/3, 1/3)$.
(b) $\mathbf{v}_2$: dots $(0,1,1) \to (1,3,3)/7 =$

Official answer: $(1/7, 3/7, 3/7)$.
(c) New words 4, 5 co-occur only with each other (4–4, 4–5, 5–5). Minimum extra dimensions while preserving existing relations?

Official answer: 1 (intended).
(d) **Erratum (official):** because $\mathbf{v}_1 = \mathbf{0}$, word 1 always gives a uniform distribution, so the intended "saturation" is impossible. A nearly optimal solution with 0 extra dimensions exists: $\mathbf{v}_4 = \mathbf{v}_5 = \mathbf{c}_4 = \mathbf{c}_5 = (0,-10)$. Then $\mathbf{v}_4 \cdot \mathbf{c}_{4,5} = 100$, which dominates every other dot product. The intended solution used 1 extra dimension with a similar structure, e.g. $\mathbf{v}_{4,5} = \mathbf{c}_{4,5} = (0,0,10)$, existing vectors padded with 0.

Midterm Fall 2022 Q5 (bigram skip-gram)

(a) Learn a vector $\mathbf{v}_b$ for every bigram, with unigram contexts: $P(y|b) = \dfrac{\exp(\mathbf{v}_b \cdot \mathbf{c}_y)}{\sum_{y' \in V} \exp(\mathbf{v}_b \cdot \mathbf{c}_{y'})}$.
(b) Runtime for one pair:

Official answer: $O(|V|d)$ ($|V|$ dot products of length d in the denominator).
(c) Training M examples for N epochs, each step costing E:

Official answer: $O(MNE)$.
(d) Parameters:

Official answer: $|B|d + |V|d$ (bigram vectors + context vectors).
(e) Represent a bigram as the sum of its word vectors: $P(y|b) = \dfrac{\exp((\mathbf{v}_{b_1} + \mathbf{v}_{b_2}) \cdot \mathbf{c}_y)}{\sum_{y'} \exp((\mathbf{v}_{b_1} + \mathbf{v}_{b_2}) \cdot \mathbf{c}_{y'})}$.
(f) Runtime:

Official answer: $O(|V|d)$. (g) Parameters:

Official answer: $2|V|d$, fewer than before since $|B| \gg |V|$.

Traps

- Include **every** vocabulary word in the softmax denominator, including the word itself.
- Use the e value the question gives ($e = 3$) and keep fractions exact.
- Word vectors and context vectors are *different* parameters. $P(y|x)$ uses $\mathbf{v}_x \cdot \mathbf{c}_y$, not $\mathbf{v}_x \cdot \mathbf{v}_y$.
- Skip-gram pairs are ordered and come in both directions; count them carefully at sentence edges.
- Dot product favors long vectors; cosine doesn't.
- The zero vector gives a uniform distribution regardless of the context vectors (Fall 2023 Q3 erratum).

10.7 Practice

Practice

1. Using the Fall 2023 Q3 vectors, compute the context distribution for $\mathbf{v}_3$ ($e = 3$).
2. Window $k = 1$: list all skip-gram pairs for "a b a c". How many are there?
3. Vocabulary 50,000, $d = 300$. How many skip-gram parameters? How many dot products to compute one $P(y|x)$ with full softmax vs. hierarchical softmax (approximately)?
4. $\mathbf{a} = [1, 2]$, $\mathbf{b} = [2, 4]$, $\mathbf{c} = [3, -1]$. Which pair is most similar by cosine? By dot product?

Answers

1. Dots with $\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$: $0, 1, -1 \to 1, 3, \frac{1}{3}$; sum $\frac{13}{3}$; so $(3/13,\ 9/13,\ 1/13)$.
2. (a,b),(b,a),(b,a),(a,b),(a,c),(c,a): 6 pairs $[2(1 \cdot 4 - 1) = 6]$.
3. $2 \times 50{,}000 \times 300 = 30{,}000{,}000$. Full softmax 50,000 dot products; hierarchical $\approx \log_2 50{,}000 \approx 16$.
4. Cosine: (a,b) $= 1$ (parallel). Dot: a·b $= 10$, b·c $= 2$, a·c $= 1$, so (a,b) again.

TOPIC 11

Deep Averaging Networks (Assignment 2)

Sources: Lecture 3.2.0 Applying Embeddings, Deep Averaging Networks; slides seg-21; Assignment 2 (Parts 2–3); Fall 2023 Q1(3), Q1(5–6), Q2(c–e).

In plain words

Look up each word's embedding, **average** them into one sentence vector, and feed that into a feedforward network with a softmax output. Simple and fast, and in 2015 it matched much fancier syntax-based models on sentiment. The cost: averaging throws away word order and interactions between words, so it can't tell "not bad" from "bad, not" and struggles with negation and contrast.

11.1 Model

DAN (Iyyer et al. 2015; A2 Part 2)

For input $s = (w_1, \ldots, w_n)$ with embedding function $e(\cdot)$:

$$\mathbf{a} = \frac{1}{n}\sum_{i=1}^{n} e(w_i) \in \mathbb{R}^d, \qquad \mathbf{h} = g(V\mathbf{a}), \qquad P(\mathbf{y}|s) = \text{softmax}(W\mathbf{h}) \quad \text{(one or more hidden layers)}.$$

Learnable parameters: feedforward layer weights (and biases) and the word embeddings, if not frozen. **No attention**, no W^K, no attention maps (Fall 2023 Q1(3)).

11.1.1 Three ways to use embeddings in the first layer (lecture)

1. **Learn from scratch** (random init + backprop). Works with enough data; can't help with words unseen in training. (The original DAN results used this.)
2. **Initialize with GloVe, keep frozen.** Efficient (no embedding updates), but inflexible if GloVe's geometry doesn't suit the task. Example: *good*/*bad* are close.
3. **Initialize with GloVe, fine-tune.** Backprop into the embedding layer. Often the standard choice for sentiment. Which is best depends on the task and amount of data.

11.2 Assignment 2 implementation details

- Data = A1 sentiment data, **lowercased** (GloVe has only lowercase). Embeddings: GloVe 50d or 300d (300d is graded), "relativized" to the dataset's words.
- `WordEmbeddings` indexer: **PAD = index 0, UNK = index 1**, both mapped to the zero vector by default. `get_initialized_embedding_layer(frozen=...)` returns an `nn.Embedding` initialized with GloVe.
- The module returns **log-probabilities**, trained with `NLLLoss` (or raw scores + `CrossEntropyLoss`), `zero_grad` / `backward` / `step`, Adam.
- **Batching:** pad sentences to equal length with index 0 (`padding_idx=0`). The average must exclude pads (divide by the true length, not the padded length). Your implementation: masked

sum / count of real tokens; dropout 0.2 on inputs, 100-unit ReLU hidden layer, dropout 0.3, 2-way log-softmax; Adam lr 0.001, 10 epochs; dev accuracy 81.5% (required 77%).

- **Part 3, typos** (dev-typo set; typos never in the first 3 characters or in short words): a typo makes a word miss the indexer and become UNK/zero. Option 1: spelling correction by edit distance (fast candidates needed; naive $O(|V|)$ per word is too slow). Option 2: **prefix embeddings** (first 3 characters, initialized from averages of words with that prefix, fine-tuned). Your implementation: edit-distance correction among same-3-prefix candidates, 80.6% on typo dev (required 74%).

Shape / parameter count for a DAN

Input: word indices [batch, len] $\rightarrow$ embeddings [batch, len, d] $\rightarrow$ average over len $\rightarrow$ [batch, d] $\rightarrow$ Linear $d \rightarrow h \rightarrow g \rightarrow$ Linear $h \rightarrow C \rightarrow$ (log-)softmax [batch, C]. Params $= |V|d$ (embeddings, if trained) $+ (dh + h) + (hC + C)$. Example: $|V| = 15{,}000$, $d = 300$, $h = 100$, $C = 2$: $4{,}500{,}000 + 30{,}100 + 202 = 4{,}530{,}302$. If embeddings are frozen, only 30,302 are trainable.

Worked example: DAN forward pass

Embeddings ($d = 2$): good $= [1, 2]$, not $= [-1, 0]$, movie $= [0, 1]$. Sentence "not good movie". $\mathbf{a} = \frac{1}{3}([-1, 0] + [1, 2] + [0, 1]) = [0, 1]$. Hidden: $V = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$, ReLU: $V\mathbf{a} = [1, 1]$, so $\mathbf{h} = [1, 1]$. Output $W = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$ (rows: neg, pos): scores $[0, 1]$, $P = \text{softmax} = [0.269, 0.731]$, so it predicts positive. **Order-blindness:** "good not movie" and "movie not good" give the same $\mathbf{a}$, hence the same prediction.

11.3 Derivations

- **Order-blindness, proved.** Addition is commutative, so $\frac{1}{n}\sum_i e(w_{\pi(i)}) = \frac{1}{n}\sum_i e(w_i)$ for any reordering π. Everything after the average sees the same input, so it gives the same output.
- **Gradient into the embeddings.** $\mathbf{a} = \frac{1}{n}\sum_i e(w_i)$, so $\frac{\partial \mathbf{a}}{\partial e(w_i)} = \frac{1}{n} I$ and $\frac{\partial \mathcal{L}}{\partial e(w_i)} = \frac{1}{n}\frac{\partial \mathcal{L}}{\partial \mathbf{a}}$. Every word in the sentence gets the *same* update (scaled by $1/n$). A word that appears twice gets it twice. Frozen embeddings simply skip this update.
- **Padding, derived.** A sentence of m real tokens padded to length L with zero vectors: naive mean $= \frac{1}{L}\sum_{i \le m} e(w_i) = \frac{m}{L} \cdot$ (true mean). The vector shrinks by the factor m/L, which depends on the batch. Divide by m instead.
- **UNK dilution.** One UNK (zero vector) among n tokens: mean $= \frac{1}{n}\sum_{\text{known}} e(w) = \frac{n-1}{n} \cdot$ (mean of known words).
- **Lecture's *not* example as arithmetic.** If $e(\text{not})$ and $e(\text{bad})$ both point in a "negative" direction, then *not bad* $= \frac{1}{2}(e(\text{not}) + e(\text{bad}))$ points even more negative than *bad*. An average can't flip a sign, so *not bad* comes out negative.

11.4 Limitations (exam favorite)

1. **Order is lost:** averaging is permutation-invariant, so "not bad" $\equiv$ "bad not". It can't model negation scope or composition. Lecture example: on *good / bad / not good / not bad*, the DAN treats both *not* and *bad* as negative evidence, so it predicts negative for 3 of the 4 phrases. That gets *not bad* wrong.

2. **Mixed sentiment / contrast:** positive and negative words average out. "but" clauses can't override earlier content.
3. **No word-to-word interactions:** the FFNN models interactions among *dimensions of the average*, not between specific words.
4. **Unseen/misspelled words** map to UNK (zero vector) and contribute nothing. Since the average still divides by them, they dilute the signal.
5. **Strengths:** simple, fast, robust to long-range *position* effects. Unlike an RNN, it isn't dominated by the last tokens (Fall 2023 Q1(5–6)).

Historical point: in 2015 people believed sentiment required syntactic (tree-structured) composition. The DAN matched recursive tree networks and Kim's CNN. That didn't show syntax is useless; it showed those models weren't exploiting it much yet.

11.5 Past exam problems

Midterm Fall 2023 Q1(3)

Parameters that can be learned in a DAN? I. feedforward layers; II. W^K; III. word embeddings; IV. attention map weights.

Official answer: I, III.

Midterm Fall 2023 Q1(5)–(6)

A unidirectional RNN predicts very positive for a long negative review ending "At least the dinner was excellent." Why?

Official answer: D: RNN state is heavily influenced by recent tokens. What might work better?

Official answer: I. BoW (more negative words overall), II. DAN, III. Transformer. The DAN averages over all words equally.

A2 / Fall 2023 Q2(c–e) connection

Typos break word lookup. Subword tokenization (Topic 17) or edit-distance repair restores the signal. Fall 2023 judged typo repair less fragile for typos anywhere in a word.

Traps

- The DAN has no attention or recurrence; don't list W^Q, W^K, W^V as DAN parameters.
- With padding, average only over real tokens.
- "Deep" refers to the feedforward layers after averaging; the averaging itself has no parameters.
- Frozen vs. fine-tuned embeddings changes the *trainable* parameter count.

11.6 Practice

Practice

1. Give two sentences with opposite sentiment that a DAN must classify identically.
2. A DAN uses 50-d embeddings (frozen), hidden 64, 3 classes, biases included. How many trainable parameters?
3. Why might padding tokens hurt if you naively average over the padded length?

Answers

1. "the food was good , the service was not bad" vs. "the food was not good , the service was bad" (same multiset of words). 2. $50 \cdot 64 + 64 + 64 \cdot 3 + 3 = 3{,}459$. 3. PAD vectors are zero, so dividing by the padded length shrinks the average toward 0, and the shrinkage depends on how much padding the batch has. The same sentence gets different representations in different batches.